Xebia

Model Context Protocol (MCP) Explained

Connecting AI Agents to Your Tools and Data

Eric Greene · Xebia

eric.greene@xebia.com

Agenda

- What MCP is, and why it exists
- Hosts, clients, servers, and the lifecycle
- Tools, resources, and prompts
- Ready-made servers you can use today
- Building a custom server in Python
- Security, trust boundaries, and production audits

What MCP is

- An open protocol connecting AI apps to tools and data
- JSON-RPC 2.0 messages over stdio or HTTP
- Servers describe their capabilities; hosts discover them at runtime
- Supported across VS Code, Claude, Cursor, and more
- A versioned specification; `2026-07-28` is current
- Builds on function calling rather than replacing it

How MCP compares

Ad-hoc integrations

Every app wires up every tool itself

N apps and M tools means N×M connectors

Function calling

Tools defined per model API

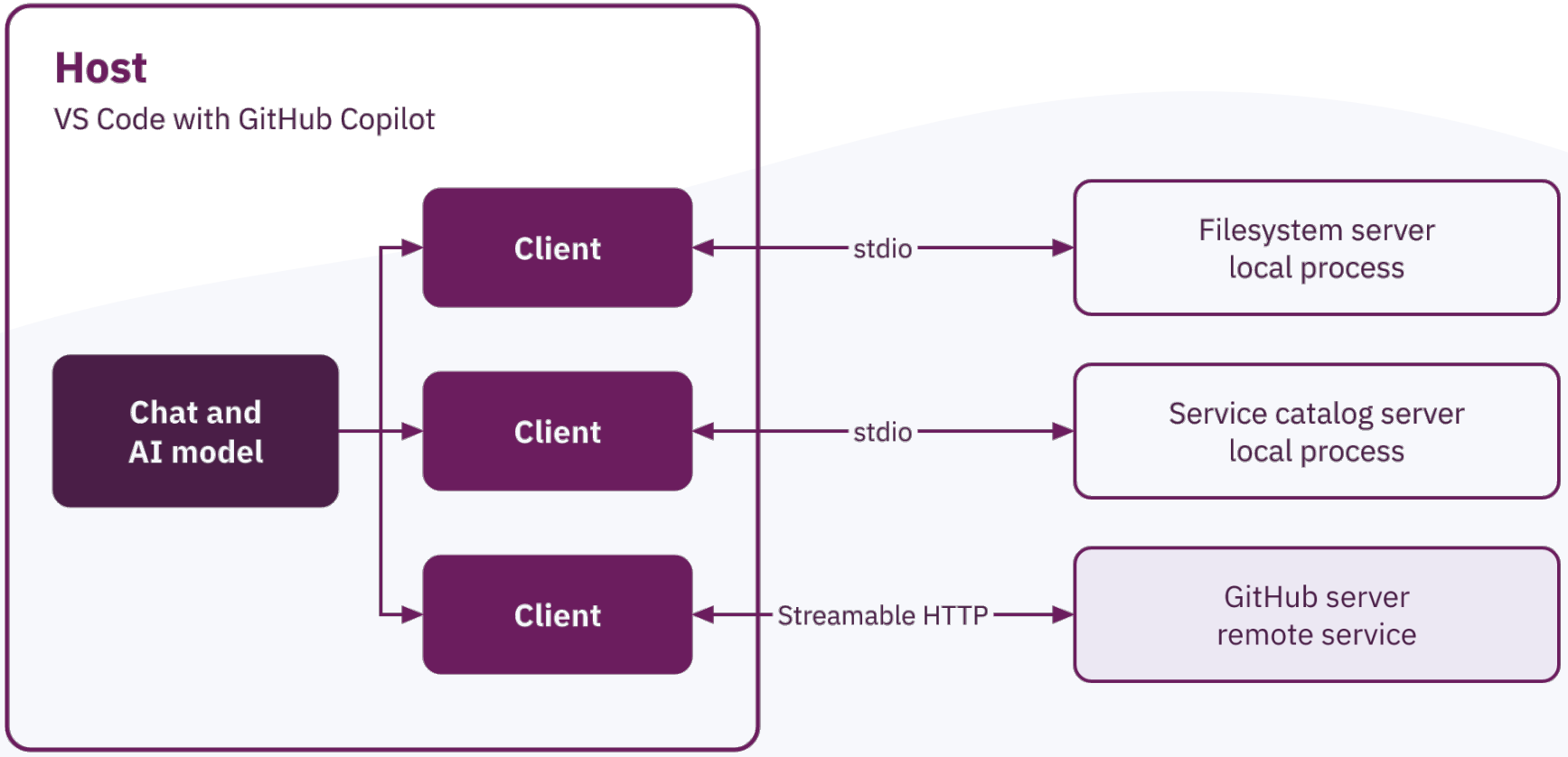
Schemas and code live inside each app

MCP

Build a server once, use it anywhere

Any MCP host discovers it at runtime

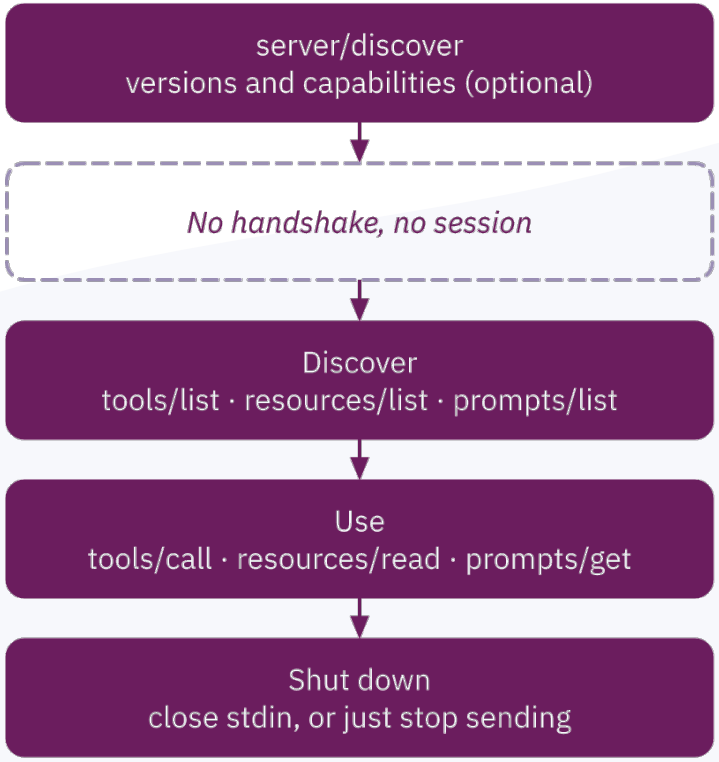
Hosts, clients, and servers



One client per server. Servers never see the model or each other.

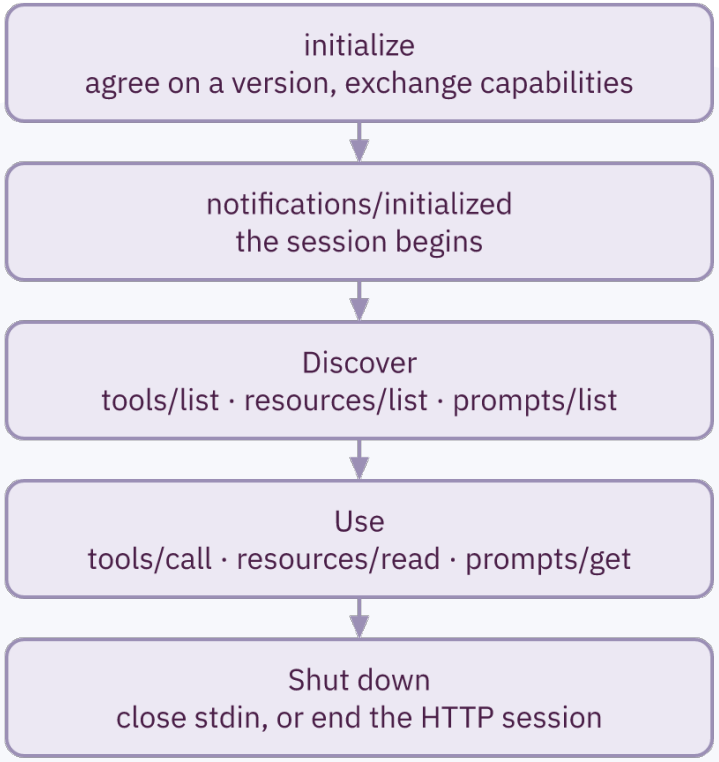
The request/response lifecycle

Modern: 2026-07-28



Every request carries the protocol version in _meta.

Legacy: 2025-11-25 and earlier



The server remembers the session after initialize.

Ready-made servers

Local, over stdio

Filesystem: read and search project files

Databases: SQLite, Postgres, and others

Git: history, diffs, and branches

Hosted, over HTTP

GitHub: issues, pull requests, and code

Microsoft Learn: current official documentation

Search and SaaS servers from vendors

Demo 1: Meet MCP

demos/01-meet-mcp · about 16 minutes

Tools, resources, and prompts

Tools

Actions the model decides to call

`search_services`, `create_incident`

Resources

Context the user or host attaches

`catalog://teams`, service runbooks

Prompts

Templates the user chooses to run

`incident_briefing`, `team_onboarding`

Choosing the right primitive

Make it a tool when

The model should decide when it's needed

It takes arguments or changes something

The answer depends on the question

Make it a resource or prompt when

The user knows which context matters

It's a document, not an action

A team wants one repeatable workflow

A tool in a few lines of Python

```
from mcp.server import MCPServer

mcp = MCPServer("service-catalog")

@mcp.tool(annotations=READ_ONLY)
def get_service(name: str) -> Service:
    """Owner, on-call, and dependencies."""
    return catalog_db.get_service(name)

mcp.run(transport="stdio")
```

Demo 2: A custom server over stdio

demos/02-custom-server-stdio · about 18 minutes

stdio or Streamable HTTP

stdio: the host starts the server

```
# server.py
mcp.run(transport="stdio")
```

```
# .vscode/mcp.json
"type": "stdio",
"command": "uv",
"args": ["run", "server.py"]
```

Streamable HTTP: the server runs at a URL

```
# http_server.py
mcp.run(transport="streamable-http",
        host="127.0.0.1", port=8765)
```

```
# .vscode/mcp.json
"type": "http",
"url": "http://127.0.0.1:8765/mcp"
```

Demo 3: The same server over HTTP

demos/03-streamable-http · about 16 minutes

Where the trust boundaries are

Model

Reads every tool description as guidance

Can be steered by injected text

Host

Approves calls and shows their arguments

Decides which servers are connected

Server

Runs code with your permissions

Reaches your files, data, and networks

Risks to plan for

- Tool poisoning: instructions hidden in tool descriptions
- Prompt injection through documents, issues, and web pages
- Over-broad access: wide roots, tokens, and write tools
- Raw query tools that run whatever the model writes
- Server updates that change tools after approval
- HTTP servers exposed without auth or Host checks

Audit before production

- Read every tool description, and pin server versions
- Grant the narrowest root, scopes, and toolsets
- Prefer read-only endpoints and parameterized tools
- Keep approvals on for anything that writes
- Require OAuth, and bind HTTP servers narrowly
- Log tool calls so you can review them

Demo 4: Security and trust boundaries

demos/04-security · about 20 minutes

Next steps

- Clone the repo and run the four demos
- Connect one ready-made server to your daily work
- Wrap one internal API as a small MCP server
- Adapt the repo's review checklist for your team
- Follow the specification changelog; it moves quickly



Thank you

Eric Greene · Xebia

eric.greene@xebia.com

xebia.com